Multi-Aspect Sentiment Analysis System for Product Reviews

Aspect-Based Sentiment Classification with Deep Learning

Project Report | Natural Language Processing | Deep (Roll No. 23BIT013), B.Tech, Pandit Deendayal Energy University

1. Introduction and Project Aim

Aim

> To design, build and evaluate an end-to-end Natural Language Processing system that reads a product review, automatically identifies every product feature (aspect) the reviewer mentions, such as battery life, screen or price, and classifies the sentiment expressed towards each feature separately as positive, neutral or negative, and to make the system usable through a web application.

The real-world problem. Shoppers and manufacturers face thousands of free-text reviews for every product. A star rating, or a single positive/negative label, hides what matters: the same laptop can have an excellent screen and a poor battery. This project converts raw reviews into feature-level insight, showing what customers praise and what they criticise.

Scope and Relevance

Ordinary rule-based programming (keyword lists and word counting) is not enough for this task, which is why NLP and computational linguistics were necessary:

- **Many ways to say one thing.** “Battery”, “battery life”, “charge” and “power” all point to one feature. A fixed keyword list cannot cover every phrasing.
- **Meaning depends on context.** “Not good” is the opposite of “good”, and “the battery dies in two hours” is negative without a single sentiment word. Counting positive and negative words fails on both.
- **One sentence, several opinions.** In “great screen but poor keyboard”, each opinion must be linked to the right feature. This needs grammar-aware analysis: part-of-speech tags, word order and attention.
- **Learning generalises.** A model trained on labelled examples copes with phrasing it has never seen. In our tests a rule-based sentiment lexicon reached only 0.475 macro-F1, against 0.57–0.61 for the trained models.

Scope: English laptop reviews, analysed sentence by sentence, with three sentiment classes. Sarcasm, implicit aspects and other languages are outside this version.

System Overview

The system is a two-stage pipeline wrapped in a web application:

1. **Data preparation:** benchmark reviews are parsed, cleaned and split into training, validation and test sets.
2. **Aspect extraction:** a BiLSTM-CRF tagger marks which words are product features.
3. **Feature representation:** context windows, TF-IDF and GloVe word embeddings turn text into numbers.
4. **Aspect sentiment classification:** a BiLSTM with aspect attention judges each feature; six other models serve as benchmarks.
5. **Evaluation:** precision, recall, F1, cross-validation and error analysis.
6. **Deployment:** a Streamlit web app highlights each aspect in colour with its sentiment and confidence.

Tools and libraries	Python, spaCy, NLTK, scikit-learn, PyTorch (with pytorch-crf), GloVe, VADER, pandas, NumPy, Matplotlib, Streamlit, Google Colab (GPU)
Core models	BiLSTM-CRF (aspect extraction); BiLSTM with aspect attention (sentiment classification)
Key results	Aspect extraction F1 = 0.74; sentiment accuracy = 63.8%, macro-F1 = 0.60 (rule-based baseline: 0.475)

2. The Data Pipeline and Preprocessing Methodology

Data Collection

The data is the **SemEval-2014 Task 4 (Aspect-Based Sentiment Analysis) Laptops** set, a standard benchmark of laptop-review sentences annotated by humans. In its initial state it is raw XML: each sentence lists its aspect terms with character positions and a sentiment label.

- **Size:** 3,045 sentences and 2,313 aspect–sentiment pairs, after removing 45 ambiguous "conflict" labels. The corpus has about 45,000 tokens and 4,597 distinct words.
- **Initial state:** 1,583 sentences (52%) mention no aspect at all. Each of the others carries one to five aspects. Class balance is 42.7% positive, 37.4% negative and only 19.9% neutral, so the data is imbalanced.
- **Quality check:** every aspect's character position was verified against the sentence text (2,313 of 2,313 matched).

Text Preprocessing Pipeline

Step	What it does, and how we used it
Tokenisation	**Splitting text into pieces (tokens).** "I don't like it" becomes I | do | n't | like | it. We used spaCy, which handles contractions and punctuation far better than splitting on spaces.
Text cleaning	**Tidying the text.** URLs, HTML tags and extra spaces are removed, words are lowercased, and punctuation and numbers are dropped for the classical models. The original text is kept for aspect extraction, because cleaning shifts character positions.
Stop-word removal	**Filtering filler words** such as "the", "and" and "of". Our twist: negation and contrast words (not, no, never, n't, but, however, very, too) are kept, because deleting "not" would turn "not good" into "good".
Lemmatisation	**Reducing words to their dictionary root:** "batteries" becomes "battery" and "was" becomes "be". We chose it over stemming, which produces non-words ("batteri", "wa").

Preprocessing in action. Original: "I charge it at night and skip taking the cord with me because of the good battery life." Cleaned tokens: charge | night | skip | take | cord | good | battery | life.

Feature Vectorisation

Machine-learning models read numbers, not words, so the cleaned text was converted in three complementary ways:

- **TF-IDF** is a word-importance score: high for words frequent in a text but rare overall. It was computed on one- to three-word sequences taken from a ±5-word window around each aspect, plus opinion words found through grammatical (dependency) analysis, giving a 1,622 × 2,914 matrix fitted on training data only. It feeds the classical models.
- **Word embeddings (GloVe, 100 numbers per word)** place similar words close together ("fast" near "quick"). They were pre-trained on 6 billion words and cover 98.8% of the words in our data; the rest are mostly typos and brand names. A Word2Vec model trained on our 2,000 sentences alone gave poor neighbours ("slow" was close to "detect"), which confirms the value of pre-trained vectors. Embeddings and part-of-speech tags feed the neural networks.
- **BIO labels** mark each word as Beginning, Inside or Outside an aspect ("battery" = B, "life" = I, "is" = O). Sentences were padded to 60 words and aspects to 5 words so batches have a fixed shape.

3. Model Architecture and Evaluation Performance

Model Selection

- **Stage 1, aspect extraction: BiLSTM-CRF.** A bidirectional LSTM reads each sentence both left-to-right and right-to-left, so every word is understood with its full context. A CRF layer then picks the best label sequence for the whole sentence, preventing impossible outputs. Part-of-speech tags are added as input because aspects are mostly nouns.
- **Stage 2, sentiment: BiLSTM with aspect attention (core model).** The aspect's vector is attached to every word and used to score attention, so the model focuses on the words that matter for that particular aspect. The same sentence can therefore receive different answers for different features.
- **Benchmarks:** a rule-based lexicon (VADER), Naive Bayes, linear SVM and Logistic Regression on TF-IDF, a CNN, and a BiLSTM without aspect information.

Training Process

- **Data split:** 70% training, 15% validation and 15% test, divided **by sentence**, so a sentence with several aspects never appears in two parts and inflates the scores. This gives 1,622 / 348 / 343 aspect pairs.
- **How the model learned:** it sees batches of 32 examples, predicts, measures its error (for the sentiment models a class-weighted loss, so the rare neutral class is not ignored) and an optimiser adjusts its weights. After every pass the validation score is checked, and training stops early when it stops improving, which guards against memorising the training data. The test set was used only once, at the end.
- **Setup:** classical models were tuned on validation data and checked with 5-fold cross-validation; the sentiment models were compared on Google Colab (GPU) and the aspect extractor was trained on a local computer.

Evaluation Metrics

Task (343 test aspects)	Accuracy	Precision	Recall	F1-score
Stage 1: aspect extraction (exact match)	n/a*	0.738	0.746	0.742
Stage 2: aspect sentiment (macro-average)	0.638	0.602	0.601	0.601

**Accuracy is not meaningful for extraction because most words are "outside" an aspect. Stage 2 precision, recall and F1 are averaged equally over positive, negative and neutral.*

Metric	What it proves in plain words
Accuracy	Out of every 100 aspects, about 64 receive the correct sentiment.
Precision	When the system flags an aspect or a sentiment, this is how often it is right, i.e. how far its flags can be trusted.
Recall	Of all the true aspects or sentiments present, this is how many the system actually finds, i.e. how little it misses.
F1-score	One number balancing precision and recall; macro-averaging gives each class equal weight, so weakness on the rare neutral class cannot hide.

Sentiment model (test set)	Accuracy	Macro-F1
Rule-based lexicon (VADER)	0.499	0.475
Naive Bayes (TF-IDF)	0.636	0.565
Linear SVM (TF-IDF)	0.656	0.595
Logistic Regression (TF-IDF)	0.647	0.608
BiLSTM + aspect attention (deployed)	**0.638**	**0.601**

Every trained model beats the rule-based baseline by about 20–28%. Differences between the trained models are within test-set noise (343 rows, roughly ±0.03). Aspect extraction: F1 0.742 against 0.316 for a simple "every noun is an aspect" rule.

4. Critical Review, Limitations and Conclusion

System Limitations

- **Mixed-opinion sentences are the main weakness.** In “great battery but poor screen” the model can give one aspect the sentiment of its neighbour. Accuracy is about 0.47–0.50 on such sentences, against about 0.70 when all aspects agree.
- **The neutral class is hardest** (F1 about 0.41, against 0.66 for negative and 0.73 for positive); factual statements and mild opinions are often mistaken for negative ones.
- **Small, single-domain data.** About 2,300 labelled aspects from laptop reviews only, so other products will be less accurate. With 343 test rows, differences smaller than about 0.03 are not meaningful, and deep models do not clearly beat simpler TF-IDF models at this scale.
- **Language phenomena:** sarcasm, slang and implicit aspects (“it dies after two hours”, with no word “battery”) are not handled. About 1% of words, mostly typos and brand names, are unknown to the pre-trained vectors and are mapped to a generic “unknown” token.
- **Boundary errors in extraction:** about one in four extracted aspects does not match the annotated wording exactly (precision 0.74), although partial-overlap F1 is 0.83.
- **Compute:** experiments used modest hardware (a local computer and a free Colab GPU), so only small models and limited hyper-parameter searches were explored.

Future Scope

- **Position-aware attention** that down-weights words far from the aspect, aimed directly at the mixed-sentence weakness.
- **Fine-tuning a pre-trained transformer** (a BERT sentence-pair classifier) for the sentiment stage, typically the largest single gain on this task.
- **Scaling the data:** add the restaurant reviews, label reviews from e-commerce sites and adapt the model to other product types and languages.
- **A joint model** that extracts aspects and classifies sentiment with one shared encoder, plus detection of implicit aspects.
- **Live deployment:** host the web application publicly, accept batch uploads of reviews and add per-product dashboards for sellers.

Conclusion

The project’s aim was to build and evaluate a system that finds the features mentioned in a review and judges the sentiment of each one. That aim was achieved in full: the delivered pipeline extracts aspects with an F1 of 0.74 (against 0.32 for a rule-based baseline), classifies each aspect’s sentiment with 63.8% accuracy and a macro-F1 of 0.60 (20–28% above the rule-based baseline), and is available as a working web application that highlights every aspect with its sentiment and confidence. The evaluation was deliberately honest: sentiment accuracy is moderate, and the analysis identified exactly where the system fails, on sentences that mix opinions about different features. That finding gives the clear next step for improvement, so the project delivers both a working tool and a reliable understanding of its limits.